

DOCKER

Complete Deployment Notes

DevOps Assignment – Group 2

Topic: Application Deployment Using Docker

A step-by-step, topic-wise guide covering Docker concepts, commands, Dockerfiles, networking, storage, Compose, registries, CI/CD, security, troubleshooting, and practical deployment.

Section	What you will learn
1	Docker fundamentals and terminology
2	Docker architecture and workflow
3	Installation and first container
4	Images and containers
5	Dockerfile and image building
6	Application deployment and ports
7	Volumes and persistent storage
8	Networking
9	Docker Compose
10	Docker Hub / registries
11	CI/CD and production deployment
12	Security and best practices
13	Troubleshooting
14	Command cheat sheet and viva questions

1. Docker Fundamentals

1.1 What is Docker?

Docker is a platform for developing, packaging, and running applications in **containers**. A container packages an application together with the libraries and configuration it needs, helping it run consistently across environments.

1.2 Why Docker is used for deployment

- **Consistency:** the same packaged application can be tested and deployed in different environments.
- **Portability:** a Docker image can run wherever a compatible Docker runtime is available.
- **Fast deployment:** containers generally start much faster than full virtual machines.
- **Isolation:** applications can run in separate containers with controlled resources and networking.
- **Reproducibility:** Dockerfiles can describe how an image is built.
- **Scalability:** containers fit well with microservices and orchestrators.

1.3 The 'works on my machine' problem

An application may work on a developer's laptop but fail on another machine because of different operating-system libraries, runtime versions, dependencies, environment variables, or configuration. Docker reduces this mismatch by packaging the application and its required environment into an image.

Simple idea: Code + Dependencies + Configuration → Docker Image → Container → Running Application

1.4 Important Docker terms

Term	Meaning
Image	Read-only template used to create containers.
Container	A running or stopped instance created from an image.
Dockerfile	Text file containing instructions for building an image.
Registry	Repository/service used to store and distribute images.
Docker Hub	A public/private container image registry service.
Volume	Docker-managed storage used to persist data.
Network	Mechanism that allows containers to communicate.
Compose	Tool for defining and running multi-container applications.

2. Docker Architecture

At a high level, Docker uses a client-server architecture. The Docker CLI sends commands to the Docker daemon, which manages images, containers, networks, and volumes.

Component	Role
Docker CLI / Client	Accepts commands such as docker run and docker build.
Docker Daemon	Background service that manages Docker objects and operations.
Docker Engine	Core Docker technology used to build and run containers.
Image	Packaged application template.
Container	Runtime instance of an image.

Component	Role
Registry	Stores and distributes images.
Network	Connects containers and external clients.
Volume	Provides persistent storage.

Flow: User → Docker CLI → Docker Daemon/Engine → Image/Container/Network/Volume

3. Installation & First Container

3.1 Installation

Install Docker Desktop on Windows/macOS, or install Docker Engine using the official instructions for a Linux distribution. After installation, verify that the Docker CLI can communicate with the engine.

```
docker --version
docker info
```

3.2 Download an image

```
docker pull nginx
```

The command downloads the Nginx image from a configured registry. If no registry is specified, Docker commonly uses Docker Hub.

3.3 Run the first container

```
docker run nginx
docker run -d --name my-nginx nginx
docker ps
docker ps -a
```

-d runs the container in detached/background mode. **--name** assigns a readable container name.

4. Docker Images

An image is an immutable package made of filesystem layers and metadata. Images are normally identified using a repository name and optional tag, such as **nginx:latest** or **my-app:1.0**.

```
docker images
docker image ls
docker pull nginx:latest
docker image inspect nginx
docker image rm nginx:latest
```

4.1 Image vs Container

Image	Container
Template/package	Runtime instance
Usually immutable	Has a writable container layer
Stored locally or in a registry	Created and managed by Docker
Used to create containers	Runs the application

One image can be used to create many containers.

5. Container Lifecycle & Management

A common lifecycle is: **create** → **start** → **running** → **stop** → **start again** → **remove**.

```
docker create --name web nginx
docker start web
docker stop web
docker restart web
docker rm web
docker ps
docker ps -a
```

5.1 Logs and shell access

```
docker logs my-nginx
docker logs -f my-nginx
docker exec -it my-nginx sh
```

docker logs helps inspect application output. **docker exec** runs a command inside an already-running container.

6. Dockerfile & Image Building

A Dockerfile is a text file containing ordered instructions used to build an image. Each instruction contributes to the resulting image and its metadata.

6.1 Example Dockerfile

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

6.2 Meaning of the instructions

Instruction	Purpose
FROM	Selects the base image.
WORKDIR	Sets the working directory.
COPY	Copies files from build context into the image.
RUN	Executes a command while building the image.
EXPOSE	Documents the port the application listens on.
CMD	Defines the default command when a container starts.

6.3 Build an image

```
docker build -t my-app:1.0 .
docker images
```

The final dot (.) tells Docker to use the current directory as the build context. The tag **my-app:1.0** gives the image a repository/name and version.

7. Application Deployment with Docker

A basic deployment packages an application into an image, then starts a container from that image and publishes the required application port.

7.1 Run a custom application

```
docker run -d -p 3000:3000 --name my-app my-app:1.0
```

Port mapping: host port : container port. Here, requests arriving at host port 3000 are forwarded to port 3000 in the container.

Browser → Host:3000 → Docker port mapping → Container:3000 → Application

7.2 Verify deployment

```
docker ps
docker logs my-app
docker logs -f my-app
docker exec -it my-app sh
```

7.3 Stop/remove

```
docker stop my-app
docker rm my-app
```

8. Docker Volumes & Persistent Storage

Containers are disposable. Data written only to a container's writable layer can disappear when the container is removed. Volumes provide a Docker-managed way to persist data independently of a particular container.

```
docker volume ls
docker volume create app-data
docker volume inspect app-data
docker volume rm app-data
```

8.1 Mount a volume

```
docker run -d -v app-data:/data --name data-container IMAGE
```

The volume **app-data** is mounted at **/data** inside the container. The exact path depends on the application.

Use volumes for: databases, uploaded files, application state, and other data that must survive container replacement.

9. Docker Networking

Docker networking lets containers communicate with one another and with external systems. User-defined networks are commonly used to connect application components.

```
docker network ls
docker network create app-network
docker network inspect app-network
docker network rm app-network
```

9.1 Connect a container to a network

```
docker run -d --name backend --network app-network backend-image
docker run -d --name frontend --network app-network frontend-image
```

On a user-defined network, containers can communicate using container/service names and the appropriate ports. Avoid exposing internal-only services to the host unless needed.

10. Docker Compose

Docker Compose is used to define and run applications made of multiple services. A Compose file can describe services, networks, volumes, images/build instructions, environment variables, and ports.

10.1 Typical architecture

Frontend → Backend/API → Database

10.2 Example compose.yaml

```
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
```

```

    - backend

backend:
  build: ./backend
  ports:
    - "5000:5000"
  depends_on:
    - db

db:
  image: mysql:8
  environment:
    MYSQL_ROOT_PASSWORD: example
    MYSQL_DATABASE: appdb
  volumes:
    - db-data:/var/lib/mysql

volumes:
  db-data:

```

Note: This is a teaching example. In real deployments, use secure secret handling and appropriate database configuration.

10.3 Compose commands

```

docker compose up
docker compose up -d
docker compose ps
docker compose logs
docker compose build
docker compose restart
docker compose down

```

11. Docker Hub & Container Registries

A registry stores container images so they can be shared between development, CI/CD systems, and deployment servers.

```

docker login
docker tag my-app:1.0 username/my-app:1.0
docker push username/my-app:1.0
docker pull username/my-app:1.0

```

A typical flow is: **Build locally/CI** → **Tag image** → **Push to registry** → **Pull on deployment server** → **Run container**.

12. Complete Deployment Workflow

Step 1: Create application code and Dockerfile.

Step 2: Build the image.

```
docker build -t my-app:1.0 .
```

Step 3: Test locally.

```
docker run -d -p 3000:3000 --name my-app my-app:1.0
```

Step 4: Check container and logs.

```

docker ps
docker logs my-app

```

Step 5: Tag and push the image.

```

docker tag my-app:1.0 username/my-app:1.0
docker push username/my-app:1.0

```

Step 6: On the deployment server, pull the image.

```
docker pull username/my-app:1.0
```

Step 7: Start the deployed container.

```
docker run -d -p 3000:3000 --name my-app username/my-app:1.0
```

Final workflow: CODE → DOCKERFILE → BUILD → IMAGE → REGISTRY → PULL → CONTAINER → DEPLOYED APPLICATION

13. Docker in CI/CD

Docker fits into CI/CD by making the application build artifact a container image. A pipeline can automatically build, test, scan, publish, and deploy that image.

Example: GitHub → CI runner → Docker build → tests/scan → registry → deployment server → container

- Jenkins can build and deploy Docker images.
- GitHub Actions can automate Docker build/push workflows.
- GitLab CI/CD can build, test, publish, and deploy containers.
- Azure DevOps can automate container image pipelines and deployments.

Important: CI/CD tools automate the process; Docker provides the packaging/runtime mechanism.

14. Docker Security & Best Practices

- Use trusted and maintained base images.
- Pin versions/tags when reproducibility matters instead of relying blindly on floating tags.
- Keep images and dependencies updated.
- Run containers with the least privilege necessary; avoid unnecessary root privileges.
- Do not hard-code passwords, API keys, or tokens in Dockerfiles or images.
- Use secrets/environment configuration appropriately.
- Expose only required ports.
- Scan images for known vulnerabilities.
- Use a small, appropriate base image where practical.
- Add a .dockerignore file to keep unnecessary files out of the build context.

14.1 Example .dockerignore

```
node_modules
.git
.env
npm-debug.log
Dockerfile
docker-compose.yml
```

Do not treat the example as universal; choose entries based on your project.

15. Troubleshooting

Problem	Useful checks
Container exits	docker ps -a; docker logs CONTAINER
Port already in use	Check host port usage; map a different host port.
Image not found	Check image name/tag; docker images; docker pull IMAGE

Problem	Useful checks
Cannot connect to app	Check docker ps, logs, port mapping, and application bind address.
Build fails	Read the failing Dockerfile step; verify files in build context.
Container cannot reach another service	Check network membership, service/container name, and internal port.
Data disappears	Use a volume or another persistent storage mechanism.
Permission error	Check user/permissions inside the image and mounted directories.

15.1 Useful inspection commands

```

docker inspect CONTAINER
docker image inspect IMAGE
docker network inspect NETWORK
docker volume inspect VOLUME
docker stats
docker system df

```

16. Essential Docker Command Cheat Sheet

Task	Command
Version	docker --version
Pull image	docker pull IMAGE
List images	docker images
Run container	docker run IMAGE
Run in background	docker run -d IMAGE
Name container	docker run --name NAME IMAGE
Map port	docker run -p HOST:CONTAINER IMAGE
List running containers	docker ps
List all containers	docker ps -a
Start	docker start CONTAINER
Stop	docker stop CONTAINER
Restart	docker restart CONTAINER
Remove container	docker rm CONTAINER
Logs	docker logs CONTAINER
Follow logs	docker logs -f CONTAINER
Shell	docker exec -it CONTAINER sh
Build image	docker build -t NAME:TAG .
Remove image	docker image rm IMAGE
List volumes	docker volume ls
Create volume	docker volume create NAME
List networks	docker network ls

Task	Command
Create network	docker network create NAME
Login	docker login
Tag	docker tag SOURCE TARGET
Push	docker push IMAGE
Compose up	docker compose up -d
Compose status	docker compose ps
Compose logs	docker compose logs
Compose down	docker compose down

17. Key Advantages & Limitations

Advantages

- Fast and repeatable application deployment.
- Consistent application environments.
- Portable packaging.
- Efficient resource use compared with full virtual machines in many workloads.
- Easy integration with CI/CD.
- Supports microservices and isolated application components.

Limitations / Challenges

- Container security still requires careful configuration.
- Persistent storage and networking require planning.
- Large-scale deployments may require an orchestrator such as Kubernetes.
- Not every workload benefits equally from containers.

18. Docker vs Virtual Machines

Aspect	Docker Containers	Virtual Machines
Virtualization	OS-level isolation	Hardware virtualization
Typical size	Usually smaller	Usually larger
Startup	Usually fast	Usually slower
OS	Share host kernel model	Each VM has its own guest OS
Use	Apps, services, CI/CD	Full OS isolation and legacy workloads

Containers and VMs are not direct replacements in every situation. They solve related but different deployment/isolation needs.

19. Practical Mini Deployment

Assume a Node.js application listens on port 3000.

Step 1 — Dockerfile

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Step 2 — Build

```
docker build -t my-app:1.0 .
```

Step 3 — Run

```
docker run -d -p 3000:3000 --name my-app my-app:1.0
```

Step 4 — Verify

```
docker ps
docker logs my-app
```

Step 5 — Stop and remove

```
docker stop my-app
docker rm my-app
```

This demonstrates the core deployment lifecycle: **package** → **build** → **run** → **verify** → **manage**.

20. Viva / Exam Questions

Q: What is Docker?

A: A platform for packaging and running applications in containers.

Q: What is a Docker image?

A: A packaged, immutable template used to create containers.

Q: What is a container?

A: A runtime instance created from a Docker image.

Q: What is a Dockerfile?

A: A text file containing instructions to build an image.

Q: What does docker build do?

A: Builds a Docker image from a Dockerfile and build context.

Q: What does docker run do?

A: Creates and starts a container from an image.

Q: What is -p in docker run?

A: Publishes/maps a host port to a container port.

Q: Why use volumes?

A: To persist data independently of a container's lifecycle.

Q: What is Docker Compose?

A: A tool for defining and running multi-container applications.

Q: What is Docker Hub?

A: A registry service for storing and distributing container images.

Q: What is a registry?

A: A service/repository for storing and distributing container images.

Q: Docker image vs container?

A: Image is the template; container is its runtime instance.

Q: Why is Docker useful in deployment?

A: It provides portable, consistent, repeatable application packaging and execution.

Q: What is CI/CD with Docker?

A: Automation that can build, test, publish, and deploy container images.

Q: Why not put secrets in a Dockerfile?

A: They can become exposed through source code or image layers; use secure secret/configuration mechanisms.

21. Final Summary

Docker simplifies application deployment by packaging software and its dependencies into images and running those images as containers. The core practical workflow is:

Code → Dockerfile → docker build → Image → docker run → Container → Application

For distributed applications, add **networks**, **volumes**, and **Docker Compose**. For automated delivery, connect Docker to **CI/CD** and a **container registry**. For production, apply security, monitoring, backup, and operational best practices.

Key takeaway: Docker makes deployment more consistent, repeatable, portable, and easier to automate.

End of Notes — DevOps Assignment, Group 2